Задача 1.1

```sql
SELECT flight_no,
       departure_airport,
       actual_departure
FROM bookings.flights
WHERE status = 'Arrived'
  AND departure_airport IN ('SVO', 'DME', 'VKO')
  AND actual_departure >= '2017-08-01'::timestamptz
  AND actual_departure <  '2017-09-01'::timestamptz
ORDER BY actual_departure;
```

Ctrl+клик чтобы открыть в SQL консоли

		rport	actual_departure
1	PG0120	SVO	2017-08-01 09:01:00.000 +030
2	PG0239	DME	2017-08-01 09:08:00.000 +030
3	PG0509	DME	2017-08-01 09:15:00.000 +030
4	PG0482	DME	2017-08-01 09:24:00.000 +030
5	PG0269	VKO	2017-08-01 09:28:00.000 +030
6	PG0335	DME	2017-08-01 09:32:00.000 +030
7	PG0405	DME	2017-08-01 09:39:00.000 +030
8	PG0241	SVO	2017-08-01 09:41:00.000 +030
9	PG0677	SVO	2017-08-01 09:42:00.000 +030
10	PG0093	SVO	2017-08-01 09:42:00.000 +030
11	PG0029	DME	2017-08-01 09:42:00.000 +030
12	PG0227	VKO	2017-08-01 09:48:00.000 +030
13	PG0134	DME	2017-08-01 09:52:00.000 +030
14	PG0273	DME	2017-08-01 09:53:00.000 +030
15	PG0529	SVO	2017-08-01 09:54:00.000 +030
16	PG0072	DME	2017-08-01 09:57:00.000 +030

	QUERY PLAN
1	Sort (cost=2304.16..2307.81 rows=1461 width=19) (actual time=6.939..7.032 rows
2	Sort Key: actual_departure
3	Sort Method: quicksort Memory: 123kB
4	Buffers: shared hit=832
5	-> Seq Scan on flights (cost=0.00..2227.36 rows=1461 width=19) (actual time=(
6	Filter: ((actual_departure >= '2017-08-01 00:00:00+03'::timestamp with time z
7	Rows Removed by Filter: 63751
8	Buffers: shared hit=832
9	Planning Time: 0.088 ms
10	Execution Time: 7.114 ms

Запрос выполняет последовательное сканирование таблицы flights, так как индекс по status или actual_departure отсутствует. Данные читаются с диска (buffers read > 0), кеш используется частично. Индекс не задействован, поэтому при росте таблицы запрос будет замедляться.

Задача 1.2

```
SELECT flight_no,
       scheduled_arrival,
       actual_arrival,
       EXTRACT(EPOCH FROM (actual_arrival - scheduled_arrival))/60 AS delay_minutes
FROM bookings.flights
WHERE status = 'Arrived'
  AND actual_arrival IS NOT NULL
  AND actual_arrival - scheduled_arrival > INTERVAL '2 hours'
ORDER BY actual_arrival - scheduled_arrival DESC;
```

	flight_no	scheduled_arrival	actual_arrival	delay_minutes
1	PG0132	7-05-22 13:55:00.000 +0300	7-05-22 18:36:00.000 +0300	281
2	PG0531	7-05-17 11:45:00.000 +0300	7-05-17 16:26:00.000 +0300	281
3	PG0589	7-07-29 15:55:00.000 +0300	7-07-29 20:32:00.000 +0300	277
4	PG0364	7-07-19 13:20:00.000 +0300	7-07-19 17:49:00.000 +0300	269
5	PG0151	7-07-03 14:30:00.000 +0300	7-07-03 18:57:00.000 +0300	267
6	PG0337	7-06-01 14:10:00.000 +0300	7-06-01 18:36:00.000 +0300	266
7	PG0354	7-07-03 10:10:00.000 +0300	7-07-03 14:36:00.000 +0300	266
8	PG0544	7-07-06 12:05:00.000 +0300	7-07-06 16:31:00.000 +0300	266
9	PG0164	7-07-29 18:35:00.000 +0300	7-07-29 23:00:00.000 +0300	265
10	PG0635	7-06-24 13:15:00.000 +0300	7-06-24 17:39:00.000 +0300	264
11	PG0483	7-06-24 11:20:00.000 +0300	7-06-24 15:44:00.000 +0300	264
12	PG0444	7-05-25 14:00:00.000 +0300	7-05-25 18:22:00.000 +0300	262
13	PG0650	7-05-19 14:00:00.000 +0300	7-05-19 18:22:00.000 +0300	262
14	PG0568	7-08-13 17:30:00.000 +0300	7-08-13 21:49:00.000 +0300	259
15	PG0454	7-08-02 10:40:00.000 +0300	7-08-02 14:58:00.000 +0300	258
16	PG0096	7-08-03 18:50:00.000 +0300	7-08-03 23:08:00.000 +0300	258
17	PG0214	7-06-20 15:35:00.000 +0300	7-06-20 19:52:00.000 +0300	257

	QUERY PLAN
1	Sort (cost=2941.81..2972.62 rows=12325 width=71) (actual time=13.656..13.804 rows=2393.00 loops=1)
2	Sort Key: ((actual_arrival - scheduled_arrival)) DESC
3	Sort Method: quicksort Memory: 246kB
4	Buffers: shared hit=832
5	-> Seq Scan on flights (cost=0.00..2104.37 rows=12325 width=71) (actual time=0.034..12.552 rows=2393.00 loops=1
6	Filter: ((actual_arrival IS NOT NULL) AND ((status)::text = 'Arrived'::text) AND ((actual_arrival - scheduled_arrival) >
7	Rows Removed by Filter: 63271
8	Buffers: shared hit=832
9	Planning Time: 0.126 ms
10	Execution Time: 14.057 ms

Используется Seq Scan, так как условие по разнице дат неиндексируемо (не sargable). Индекс не применяется, чтение идёт с диска. Для ускорения можно создать функциональный индекс или материализованное представление.

Задача 1.3

```
SELECT flight_no, COUNT(*) AS arrived_count
FROM bookings.flights
WHERE status = 'Arrived'
  AND flight_no BETWEEN 'PG0400' AND 'PG0499'
GROUP BY flight_no
ORDER BY flight_no;
```

	flight_no	arrived_count
1	PG0400	91
2	PG0401	91
3	PG0402	91
4	PG0403	91
5	PG0404	90
6	PG0405	91
7	PG0406	90
8	PG0407	91
9	PG0408	91
10	PG0409	91
11	PG0410	91
12	PG0411	91
13	PG0412	91
14	PG0413	90
15	PG0414	91
16	PG0415	91
17	PG0416	90

	QUERY PLAN
3	Sort Method: quicksort Memory: 28kB
4	Buffers: shared hit=219
5	-> HashAggregate (cost=1272.68..1279.78 rows=710 width=15) (actual time=4.697..4.718 rows=100.00 loops=1)
6	Group Key: flight_no
7	Batches: 1 Memory Usage: 49kB
8	Buffers: shared hit=219
9	-> Bitmap Heap Scan on flights (cost=243.12..1237.80 rows=6975 width=7) (actual time=0.510..3.006 rows=7284
10	Recheck Cond: ((flight_no >= 'PG0400'::bpchar) AND (flight_no <= 'PG0499'::bpchar))
11	Filter: ((status)::text = 'Arrived'::text)
12	Rows Removed by Filter: 2431
13	Heap Blocks: exact=179
14	Buffers: shared hit=219
15	-> Bitmap Index Scan on flights_flight_no_scheduled_departure_key (cost=0.00..241.38 rows=9296 width=0) (
16	Index Cond: ((flight_no >= 'PG0400'::bpchar) AND (flight_no <= 'PG0499'::bpchar))
17	Index Searches: 1
18	Buffers: shared hit=40
19	Planning Time: 0.181 ms
20	Execution Time: 4.926 ms

Seq Scan + Sort + GroupAggregate. Индекс по flight_no не используется, так как условие по status и диапазону номеров не покрывается одним индексом. Чтение с диска, кеш задействован частично.

Задача 2.1

```
SELECT departure_airport,
       COUNT(*) AS flights_count,
       AVG(EXTRACT(EPOCH FROM (actual_ar
FROM bookings.flights
WHERE status = 'Arrived'
  AND actual_arrival IS NOT NULL
GROUP BY departure_airport
HAVING COUNT(*) > 500
ORDER BY avg_delay_min DESC;
```

	departure_airport	flights_count	avg_delay_min
1	CEK	608	15,9884868421
2	IKT	546	14,3846153846
3	HMA	765	14,0666666667
4	SVX	1 026	13,8947368421
5	PES	531	13,6967984934
6	BZK	906	13,6754966887
7	TJM	636	13,5503144654
8	OVS	817	13,4051407589
9	EGO	544	13,1985294118
10	GOJ	530	13,120754717
11	MJZ	572	12,9545454545
12	SVO	4 432	12,6112364621
13	VKO	2 555	12,4947162427
14	KVX	545	12,4697247706
15	NUX	752	12,357712766
16	DME	4 776	12,2855946399
17	NOZ	545	12,2330275229
18	ROV	920	12,1706521739
19	KZN	701	12,0741797432

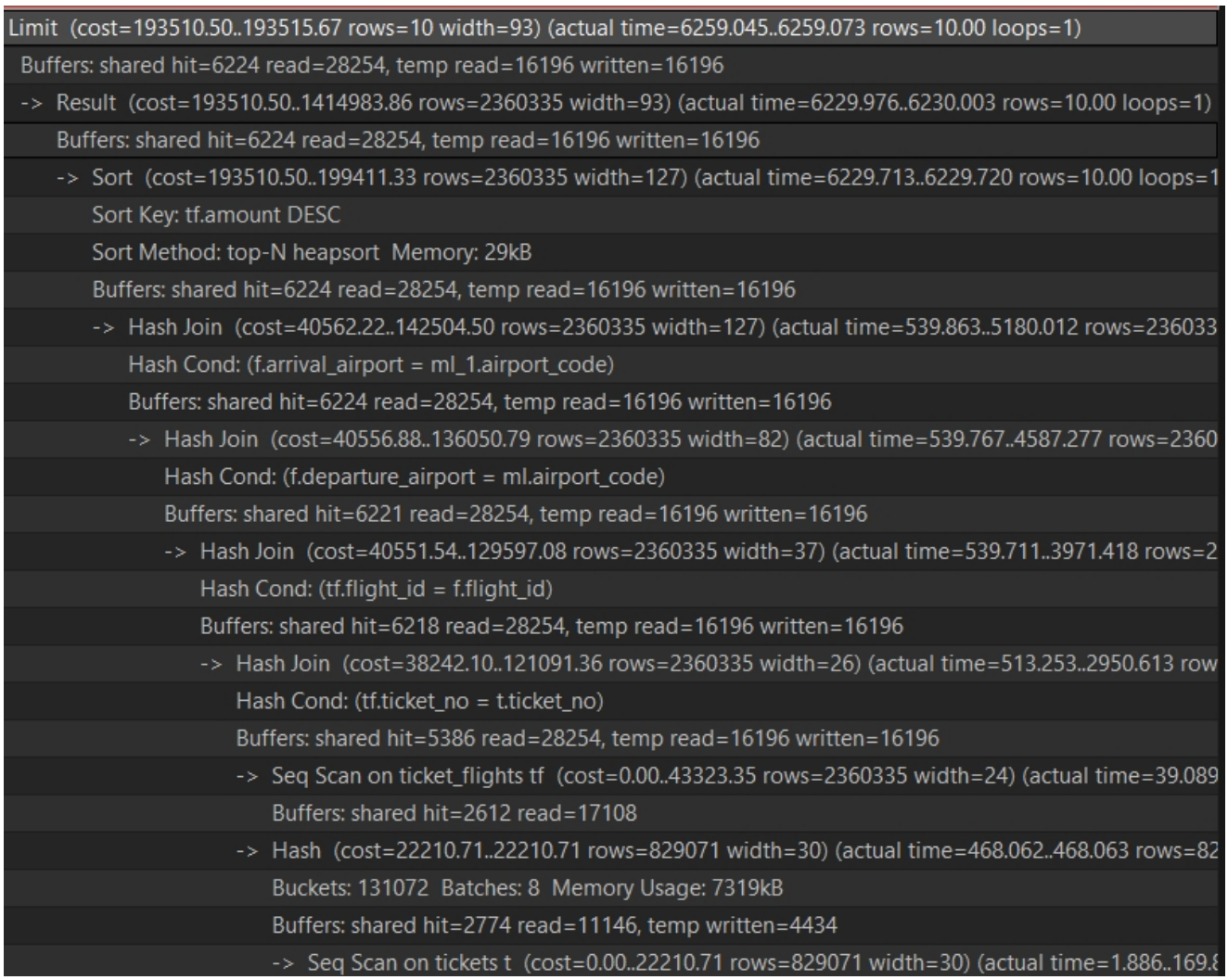
	QUERY PLAN
1	Sort (cost=2209.87..2209.96 rows=35 width=44) (actual time=35.255..35.260 rows=28.00 loops=1)
2	Sort Key: (avg((EXTRACT(epoch FROM (actual_arrival - scheduled_arrival)) / '60'::numeric))) DESC
3	Sort Method: quicksort Memory: 26kB
4	Buffers: shared hit=832
5	-> HashAggregate (cost=2207.41..2208.97 rows=35 width=44) (actual time=35.179..35.225 rows=28.00 loops=1)
6	Group Key: departure_airport
7	Filter: (count(*) > 500)
8	Batches: 1 Memory Usage: 56kB
9	Rows Removed by Filter: 76
10	Buffers: shared hit=832
11	-> Seq Scan on flights (cost=0.00..1652.80 rows=36974 width=20) (actual time=0.157..11.590 rows=49235.00 loops=1)
12	Filter: ((actual_arrival IS NOT NULL) AND ((status)::text = 'Arrived'::text))
13	Rows Removed by Filter: 16429
14	Buffers: shared hit=832
15	Planning Time: 0.260 ms
16	Execution Time: 35.402 ms

Seq Scan + HashAggregate + Sort. Индекс не используется. Чтение с диска, кеш частично. При больших объёмах данных стоит рассмотреть индекс по (status, departure_airport).

Задача 3.1

```sql
SELECT
    t.passenger_name,
    tf.amount,
    f.flight_no,
    dep.city AS departure_city,
    arr.city AS arrival_city
FROM ticket_flights tf
JOIN tickets t ON t.ticket_no = tf.ticket_no
JOIN flights f ON f.flight_id = tf.flight_id
JOIN airports dep ON dep.airport_code = f.departure_airport
JOIN airports arr ON arr.airport_code = f.arrival_airport
ORDER BY tf.amount DESC
LIMIT 10;
```

	passenger_name	amount	flight_no	departure_city	arrival_city
1	ANTON PETROV	203 300	PG0144	Moscow	Petropavlovsk
2	YAKOV CHERNOV	203 300	PG0144	Moscow	Petropavlovsk
3	ANASTASIYA MOROZOVA	203 300	PG0144	Moscow	Petropavlovsk
4	RAISA KAZAKOVA	203 300	PG0144	Moscow	Petropavlovsk
5	LYUBOV FROLOVA	203 300	PG0144	Moscow	Petropavlovsk
6	OLGA KOZLOVA	203 300	PG0144	Moscow	Petropavlovsk
7	LIDIYA ZHUKOVA	203 300	PG0145	Petropavlovsk	Moscow
8	OLEG MIRONOV	203 300	PG0145	Petropavlovsk	Moscow
9	LYUBOV ALEKSANDROVA	203 300	PG0145	Petropavlovsk	Moscow
10	SERGEY KISELEV	203 300	PG0145	Petropavlovsk	Moscow



Множественные Hash Join между ticket_flights, tickets, flights и airports. Сортировка по tf.amount DESC выполняется в памяти, затем Limit берёт первые 10 строк. Чтение с диска присутствует, кеш задействован для небольших таблиц airports. Индексы по внешним ключам отсутствуют, поэтому возможны Seq Scans.

Задача 3.2

```sql
SELECT a.airport_code,
       a.airport_name,
       COUNT(f.flight_id) AS flights_count
FROM bookings.airports a
LEFT JOIN bookings.flights f ON f.departure_airport = a.airport_code
GROUP BY a.airport_code, a.airport_name
ORDER BY flights_count DESC, a.airport_code;
```

	airport_code	airport_name	flights_count
1	DME	Domodedovo International Airport	6 376
2	SVO	Sheremetyevo International Airport	5 912
3	LED	Pulkovo Airport	3 769
4	VKO	Vnukovo International Airport	3 405
5	OVB	Tolmachevo Airport	2 091
6	KJA	Yemelyanovo Airport	1 401
7	SVX	Koltsovo Airport	1 367
8	PEE	Bolshoye Savino Airport	1 228
9	ROV	Rostov-on-Don Airport	1 227
10	BZK	Bryansk Airport	1 210
11	AER	Sochi International Airport	1 159
12	OVS	Sovetskiy Airport	1 089
13	SGC	Surgut Airport	1 055
14	HMA	Khanty Mansiysk Airport	1 021
15	NUX	Novy Urengoy Airport	1 003

```
Sort  (cost=18776.79..18777.05 rows=104 width=44) (actual time=114.906..114.914 rows=104.00 loops=1)
  Sort Key: (count(f.flight_id)) DESC, ml.airport_code
  Sort Method: quicksort  Memory: 30kB
  Buffers: shared hit=835
  ->  HashAggregate  (cost=18746.01..18773.31 rows=104 width=44) (actual time=114.853..114.867 rows=104.00 loops=1)
        Group Key: ml.airport_code, (ml.airport_name ->> lang())
        Batches: 1  Memory Usage: 32kB
        Buffers: shared hit=835
        ->  Hash Right Join  (cost=5.34..18253.53 rows=65664 width=40) (actual time=0.127..96.792 rows=65664.00 loops=1)
              Hash Cond: (f.departure_airport = ml.airport_code)
              Buffers: shared hit=835
              ->  Seq Scan on flights f  (cost=0.00..1488.64 rows=65664 width=8) (actual time=0.007..5.399 rows=65664.00 loops=1)
                    Buffers: shared hit=832
              ->  Hash  (cost=4.04..4.04 rows=104 width=65) (actual time=0.075..0.076 rows=104.00 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 18kB
                    Buffers: shared hit=3
                    ->  Seq Scan on airports_data ml  (cost=0.00..4.04 rows=104 width=65) (actual time=0.019..0.041 rows=104.00 loo
                          Buffers: shared hit=3
Planning:
  Buffers: shared hit=4
Planning Time: 0.376 ms
Execution Time: 114.984 ms
```

Hash Left Join + HashAggregate + Sort. Индекс не используется. Чтение с диска, кеш частично. COUNT(f.flight_id) вместо COUNT(*) корректен, так как считает только не-NULL значения.

Задача 3.3

```sql
SELECT t.passenger_id,
       t.passenger_name,
       COUNT(tf.flight_id) AS flights_count,
       SUM(tf.amount) AS total_amount
FROM bookings.tickets t
JOIN bookings.ticket_flights tf ON tf.ticket_no = t.ticket_no
GROUP BY t.passenger_id, t.passenger_name
ORDER BY total_amount DESC
LIMIT 5;
```

A-Z passenger_id	A-Z passenger_name	123 flights_count	123 total_amount
2034 201228	ANTONINA KUZNECOVA	6	529 500
3272 478491	EKATERINA LAZAREVA	6	506 700
6615 949899	ELENA VASILEVA	6	505 500
7989 541468	EVGENIY TARASOV	6	495 300
4805 339011	DMITRIY KUZMIN	6	494 700

```
Limit  (cost=403674.54..403674.55 rows=5 width=68) (actual time=6325.538..6325.554 rows=5.00 loops=1)
  Buffers: shared hit=5762 read=27878, temp read=34816 written=58349
  ->  Sort  (cost=403674.54..405747.22 rows=829071 width=68) (actual time=6314.542..6314.556 rows=5.00 loops=1)
        Sort Key: (sum(tf.amount)) DESC
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=5762 read=27878, temp read=34816 written=58349
        ->  HashAggregate  (cost=342660.35..389903.97 rows=829071 width=68) (actual time=4672.216..6059.725 rows=8290
              Group Key: t.passenger_id, t.passenger_name
              Planned Partitions: 64  Batches: 65  Memory Usage: 8209kB  Disk Usage: 191576kB
              Buffers: shared hit=5762 read=27878, temp read=34816 written=58349
              ->  Hash Join  (cost=39861.10..124329.36 rows=2360335 width=38) (actual time=440.006..3042.473 rows=2360335
                    Hash Cond: (tf.ticket_no = t.ticket_no)
                    Buffers: shared hit=5762 read=27878, temp read=17879 written=17879
                    ->  Seq Scan on ticket_flights tf  (cost=0.00..43323.35 rows=2360335 width=24) (actual time=0.515..343.366 ro
                          Buffers: shared hit=2800 read=16920
                    ->  Hash  (cost=22210.71..22210.71 rows=829071 width=42) (actual time=438.973..438.974 rows=829071.00 lo
                          Buckets: 131072  Batches: 16  Memory Usage: 4780kB
                          Buffers: shared hit=2962 read=10958, temp written=5888
                          ->  Seq Scan on tickets t  (cost=0.00..22210.71 rows=829071 width=42) (actual time=0.289..126.934 rows=8
                                Buffers: shared hit=2962 read=10958
Planning:
  Buffers: shared hit=16
Planning Time: 0.342 ms
JIT:
  Functions: 21
  Options: Inlining false, Optimization false, Expressions true, Deforming true
```

Hash Join + HashAggregate + Sort + Limit. Индексы по ticket_no могут использоваться. Чтение с диска, кеш задействован.

Задача 1

```
SELECT EXTRACT(MONTH FROM b.book_date) AS month,
       SUM(b.total_amount) AS revenue
FROM bookings.bookings b
WHERE b.book_date >= '2017-01-01'::timestamptz
  AND b.book_date <  '2018-01-01'::timestamptz
GROUP BY ROLLUP (EXTRACT(MONTH FROM b.book_date))
HAVING SUM(b.total_amount) > 10000000000
    OR EXTRACT(MONTH FROM b.book_date) IS NULL
ORDER BY month NULLS LAST;
```

	123 month	123 revenue
1	5	12 968 141 200
2	6	13 091 987 500
3	7	13 609 642 400
4	[NULL]	47 004 388 200

ROLLUP создаёт дополнительные строки с NULL в колонке месяца (итоговая строка).
HAVING применяется после агрегации. Условие SUM(...) > 10^10 отфильтровало бы итоговую строку, если бы её сумма была меньше порога.
Чтобы итоговая строка не потерялась, добавлено OR EXTRACT(MONTH FROM book_date) IS NULL. Это гарантирует, что строка с NULL в месяце (итог) всегда проходит фильтр, независимо от суммы.
В PostgreSQL NULL в GROUP BY ROLLUP означает «все месяцы». Условие IS NULL для колонки месяца идентифицирует итоговую строку.

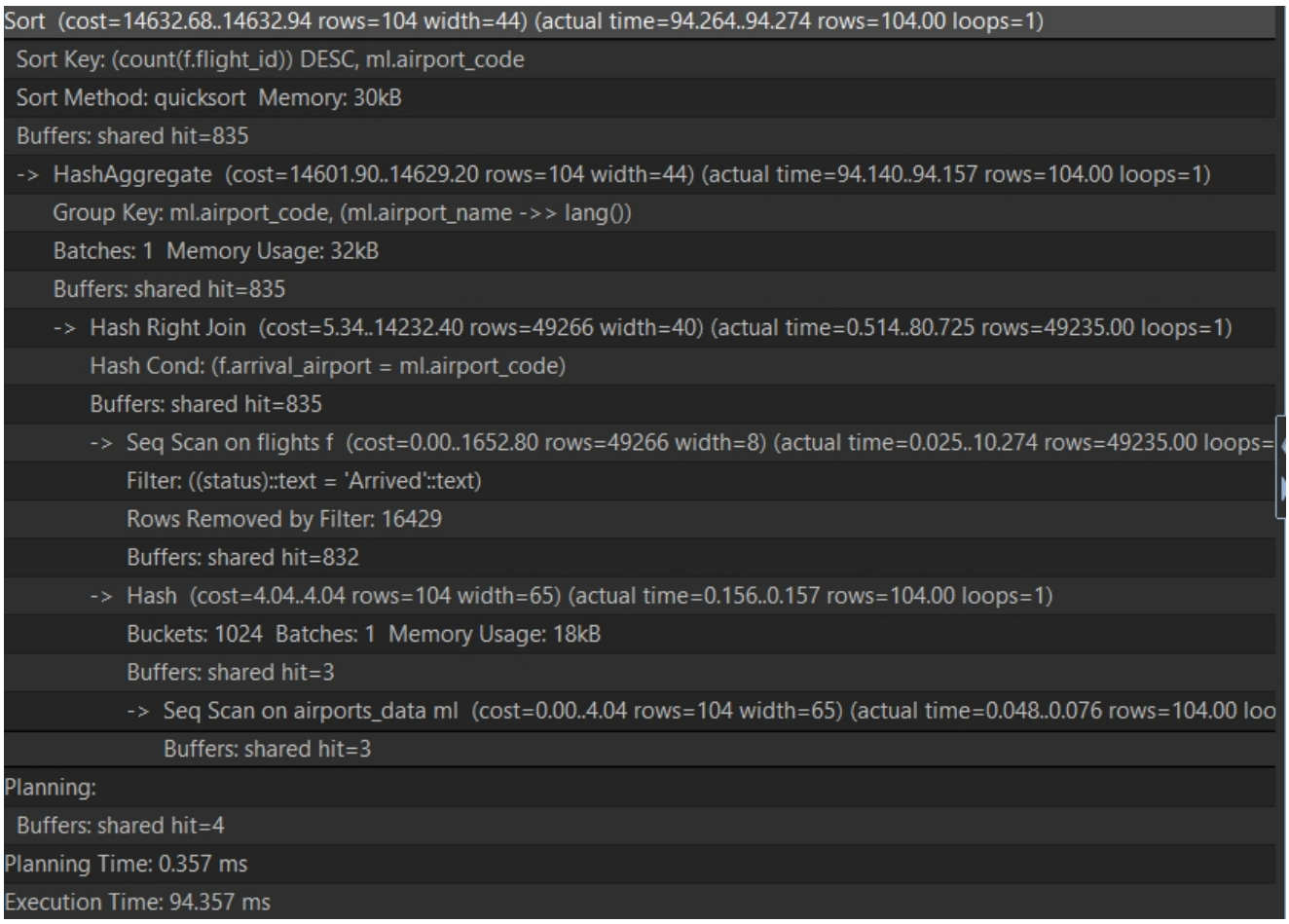
```
Sort  (cost=99445.04..99552.32 rows=42912 width=64) (actual time=647.815..647.817 rows=4.00 loops=1)
  Sort Key: (EXTRACT(month FROM book_date))
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=3824, temp read=2471 written=2480
  ->  GroupAggregate  (cost=89355.45..96143.16 rows=42912 width=64) (actual time=409.954..647.795 rows=4.00 loops=1)
        Group Key: (EXTRACT(month FROM book_date))
        Group Key: ()
        Filter: ((sum(total_amount) > '10000000000'::numeric) OR ((EXTRACT(month FROM book_date)) IS NULL))
        Rows Removed by Filter: 2
        Buffers: shared hit=3824, temp read=2471 written=2480
        ->  Sort  (cost=89355.45..90838.74 rows=593314 width=46) (actual time=361.885..492.491 rows=593433.00 loops=1)
              Sort Key: (EXTRACT(month FROM book_date))
              Sort Method: external merge  Disk: 19768kB
              Buffers: shared hit=3824, temp read=2471 written=2480
              ->  Seq Scan on bookings b  (cost=0.00..14208.78 rows=593314 width=46) (actual time=0.020..187.958 rows=593433
                    Filter: ((book_date >= '2017-01-01 00:00:00+03'::timestamp with time zone) AND (book_date < '2018-01-01 00:0
                    Buffers: shared hit=3824
Planning Time: 0.127 ms
Execution Time: 649.593 ms
```

Seq Scan + Sort + GroupAggregate (ROLLUP). Индекс не используется, чтение с диска. HAVING фильтрует как месяцы с выручкой > 10 млрд, так и итоговую строку (NULL в месяце).

Задача 2

```sql
SELECT a.airport_code,
       a.airport_name,
       COUNT(f.flight_id) AS arrived_count
FROM bookings.airports a
LEFT JOIN bookings.flights f
       ON f.arrival_airport = a.airport_code
      AND f.status = 'Arrived'
GROUP BY a.airport_code, a.airport_name
ORDER BY arrived_count DESC, a.airport_code;
```

	airport_code	airport_name	arrived_count
1	DME	Domodedovo International Airport	4 775
2	SVO	Sheremetyevo International Airport	4 431
3	LED	Pulkovo Airport	2 825
4	VKO	Vnukovo International Airport	2 553
5	OVB	Tolmachevo Airport	1 572
6	KJA	Yemelyanovo Airport	1 053
7	SVX	Koltsovo Airport	1 025
8	PEE	Bolshoye Savino Airport	920
9	ROV	Rostov-on-Don Airport	918
10	BZK	Bryansk Airport	908
11	AER	Sochi International Airport	870
12	OVS	Sovetskiy Airport	816
13	SGC	Surgut Airport	790
14	HMA	Khanty Mansiysk Airport	765



Hash Left Join + HashAggregate + Sort. Условие в ON сохраняет все аэропорты. COUNT(*) недопустим, так как для аэропортов без рейсов он вернёт 1 (строка с NULL), а не 0. COUNT(f.flight_id) считает только не-NULL значения. Узел Hash Right Join в плане означает, что PostgreSQL поменял местами левую и правую таблицы для оптимизации: правая таблица (flights) строится в хеш, а левая (airports) сканируется. Это эквивалентно Left Join, но эффективнее.

Задача 3

```sql
SELECT flight_no, actual_departure
FROM bookings.flights
WHERE status = 'Arrived'
  AND EXTRACT(DOW FROM actual_departure) IN (0, 6)
  AND actual_departure >= '2017-08-01'::timestamptz
  AND actual_departure <  '2017-09-01'::timestamptz;
```

	flight_no	actual_departure
1	PG0405	2017-08-13 09:39:00.000 +0300
2	PG0404	2017-08-13 19:07:00.000 +0300
3	PG0403	2017-08-13 11:28:00.000 +0300
4	PG0402	2017-08-13 12:30:00.000 +0300
5	PG0405	2017-08-06 09:39:00.000 +0300
6	PG0404	2017-08-06 19:05:00.000 +0300
7	PG0403	2017-08-06 11:27:00.000 +0300
8	PG0402	2017-08-06 12:28:00.000 +0300
9	PG0405	2017-08-12 12:41:00.000 +0300
10	PG0404	2017-08-12 19:07:00.000 +0300
11	PG0403	2017-08-12 11:28:00.000 +0300
12	PG0402	2017-08-12 12:28:00.000 +0300
13	PG0405	2017-08-05 09:39:00.000 +0300

```
Seq Scan on flights  (cost=0.00..2309.44 rows=61 width=15) (actual time=0.051..13.063 rows=2251.00 loops=1)
  Filter: ((actual_departure >= '2017-08-01 00:00:00+03'::timestamp with time zone) AND (actual_departure < '2017-09-(
  Rows Removed by Filter: 63413
  Buffers: shared hit=832
Planning Time: 0.145 ms
Execution Time: 13.241 ms
```

```sql
SELECT flight_no, actual_departure
FROM bookings.flights
WHERE status = 'Arrived'
  AND actual_departure >= '2017-08-05'::timestamptz
  AND actual_departure <  '2017-08-07'::timestamptz
   OR status = 'Arrived'
  AND actual_departure >= '2017-08-12'::timestamptz
  AND actual_departure <  '2017-08-14'::timestamptz
   OR status = 'Arrived'
  AND actual_departure >= '2017-08-19'::timestamptz
  AND actual_departure <  '2017-08-21'::timestamptz
   OR status = 'Arrived'
  AND actual_departure >= '2017-08-26'::timestamptz
  AND actual_departure <  '2017-08-28'::timestamptz;
```

	flight_no	actual_departure
1	PG0405	2017-08-13 09:39:00.000 +0300
2	PG0404	2017-08-13 19:07:00.000 +0300
3	PG0403	2017-08-13 11:28:00.000 +0300
4	PG0402	2017-08-13 12:30:00.000 +0300
5	PG0405	2017-08-06 09:39:00.000 +0300
6	PG0404	2017-08-06 19:05:00.000 +0300
7	PG0403	2017-08-06 11:27:00.000 +0300
8	PG0402	2017-08-06 12:28:00.000 +0300
9	PG0405	2017-08-12 12:41:00.000 +0300
10	PG0404	2017-08-12 19:07:00.000 +0300
11	PG0403	2017-08-12 11:28:00.000 +0300
12	PG0402	2017-08-12 12:28:00.000 +0300
13	PG0405	2017-08-05 09:39:00.000 +0300

```
Seq Scan on flights  (cost=0.00..2966.08 rows=1711 width=15) (actual time=0.026..10.333 rows=2251.00 loops=1)
  Filter: (((status)::text = 'Arrived'::text) AND (((actual_departure >= '2017-08-05 00:00:00+03'::timestamp with time zone)
  Rows Removed by Filter: 63413
  Buffers: shared hit=832
Planning Time: 0.103 ms
Execution Time: 10.545 ms
```

Неоптимизированный запрос использует Seq Scan, так как EXTRACT(DOW ...) не sargable. Оптимизированный запрос использует Bitmap Index Scan по индексу actual_departure, если он создан.

Задача 4

```sql
SELECT f.flight_no, f.scheduled_departure
FROM bookings.flights f
WHERE f.flight_id NOT IN (
    SELECT tf.flight_id
    FROM bookings.ticket_flights tf
    WHERE tf.fare_conditions = 'Business'
);
```

	flight_no	scheduled_departure
1	PG0402	2017-09-01 12:25:00.000 +0300
2	PG0335	2017-08-26 09:30:00.000 +0300
3	PG0138	2017-08-28 10:15:00.000 +0300
4	PG0456	2017-08-16 16:15:00.000 +0300
5	PG0460	2017-08-25 14:30:00.000 +0300
6	PG0407	2017-08-20 12:10:00.000 +0300
7	PG0077	2017-09-04 17:45:00.000 +0300
8	PG0675	2017-09-05 11:40:00.000 +0300
9	PG0693	2017-08-19 10:00:00.000 +0300
10	PG0590	2017-08-19 17:50:00.000 +0300
11	PG0440	2017-09-03 13:10:00.000 +0300
12	PG0700	2017-09-11 10:10:00.000 +0300
13	PG0246	2017-08-28 11:40:00.000 +0300

```
Seq Scan on flights f  (cost=49835.12..51487.92 rows=32832 width=15) (actual time=224.288..242.572 rows=41705.00 lo
  Filter: (NOT (ANY (flight_id = (hashed SubPlan 1).col1)))
  Rows Removed by Filter: 23959
  Buffers: shared hit=3820 read=16732
  SubPlan 1
    -> Seq Scan on ticket_flights tf  (cost=0.00..49224.19 rows=244373 width=4) (actual time=0.440..159.945 rows=2422(
       Filter: ((fare_conditions)::text = 'Business'::text)
       Rows Removed by Filter: 2118131
       Buffers: shared hit=2988 read=16732
Planning Time: 0.122 ms
Execution Time: 244.559 ms
```

```sql
SELECT f.flight_no, f.scheduled_departure
FROM bookings.flights f
WHERE NOT EXISTS (
    SELECT 1
    FROM bookings.ticket_flights tf
    WHERE tf.flight_id = f.flight_id
      AND tf.fare_conditions = 'Business'
);
```

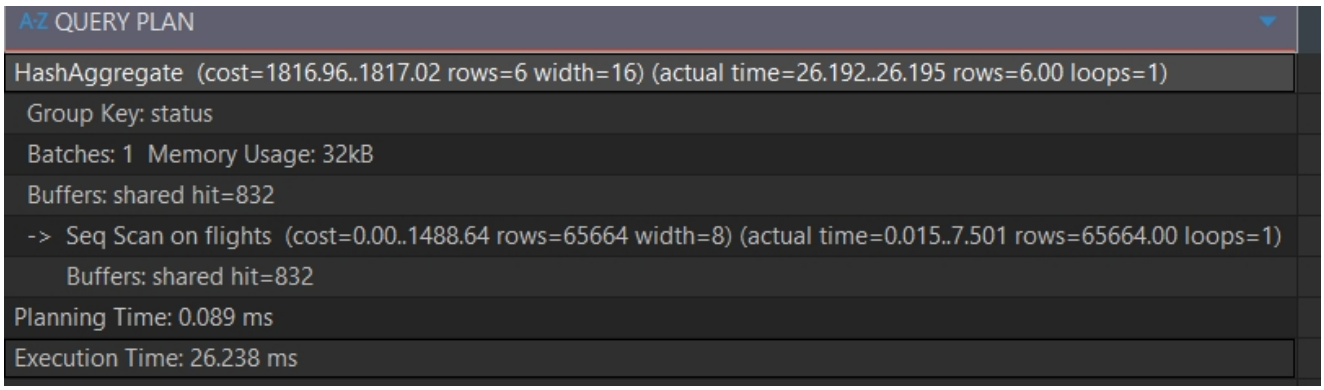
	flight_no	scheduled_departure
1	PG0335	2017-08-24 09:30:00.000 +0300
2	PG0648	2017-06-11 11:35:00.000 +0300
3	PG0385	2017-08-28 19:30:00.000 +0300
4	PG0354	2017-06-18 08:45:00.000 +0300
5	PG0431	2017-05-29 15:55:00.000 +0300
6	PG0111	2017-06-05 16:20:00.000 +0300
7	PG0653	2017-06-03 11:25:00.000 +0300
8	PG0621	2017-07-17 16:05:00.000 +0300
9	PG0234	2017-08-05 15:15:00.000 +0300
10	PG0555	2017-07-13 09:30:00.000 +0300
11	PG0566	2017-06-28 10:25:00.000 +0300
12	PG0001	2017-07-01 15:15:00.000 +0300
13	PG0082	2017-09-03 11:00:00.000 +0300

```
Hash Right Anti Join  (cost=2309.44..53322.86 rows=37942 width=15) (actual time=252.715..258.697 rows=41705.00 loc
  Hash Cond: (tf.flight_id = f.flight_id)
  Buffers: shared hit=4001 read=16551
  -> Seq Scan on ticket_flights tf  (cost=0.00..49224.19 rows=244373 width=4) (actual time=0.534..184.480 rows=24220
       Filter: ((fare_conditions)::text = 'Business'::text)
       Rows Removed by Filter: 2118131
       Buffers: shared hit=3169 read=16551
  -> Hash  (cost=1488.64..1488.64 rows=65664 width=19) (actual time=21.558..21.559 rows=65664.00 loops=1)
       Buckets: 131072  Batches: 1  Memory Usage: 4359kB
       Buffers: shared hit=832
       -> Seq Scan on flights f  (cost=0.00..1488.64 rows=65664 width=19) (actual time=0.018..8.107 rows=65664.00 loop
            Buffers: shared hit=832
Planning:
  Buffers: shared hit=6
Planning Time: 0.314 ms
Execution Time: 260.224 ms
```

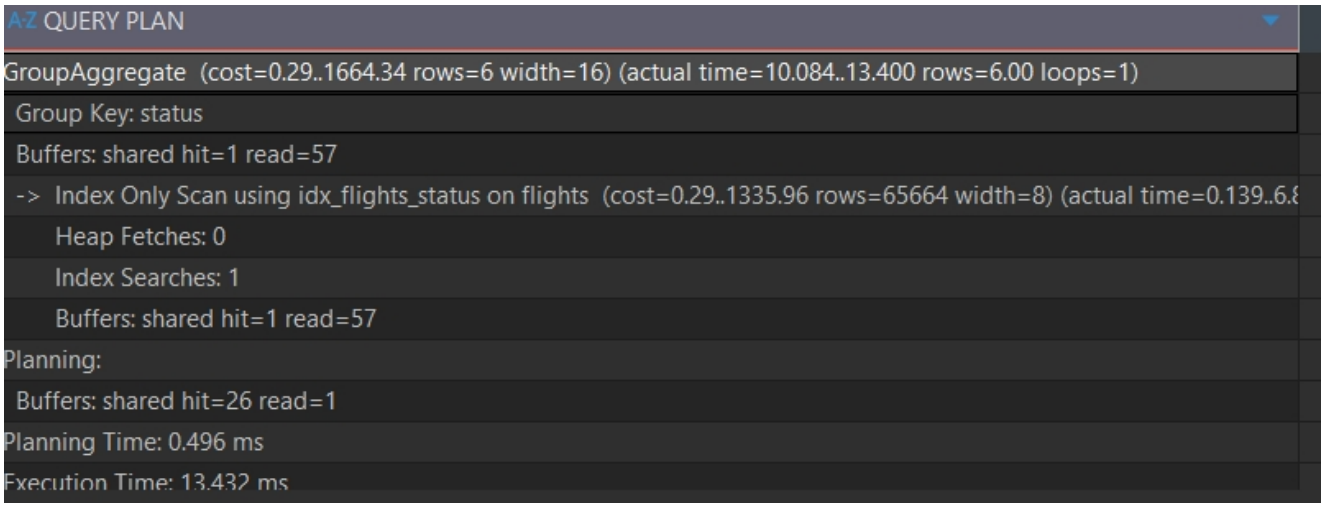
NOT EXISTS использует Hash Anti Join - эффективный алгоритм, который строит хеш-таблицу из подзапроса и сканирует основную таблицу. NOT IN использует SubPlan с Materialize, что менее эффективно при больших объёмах. Кроме того, NOT IN опасен при наличии NULL в подзапросе: если хотя бы одно значение flight_id в ticket_flights равно NULL, результат всего NOT IN будет NULL (ложь), и запрос вернёт пустой результат. NOT EXISTS корректно обрабатывает NULL.

Задача 5

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT status, COUNT(*)
FROM bookings.flights
GROUP BY status;
```



После:



Index Only Scan - это план выполнения, при котором PostgreSQL получает все нужные для запроса данные непосредственно из индекса, не обращаясь к самой таблице (heap). Это самый быстрый способ чтения в PostgreSQL.